DEVELOPING A RAPID URBAN FOREST ASSESSMENT SYSTEM FOR SUSTAINABLE CITY GREENIFICATION

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Computer Science

by

Daniel Gonzalez

June 2026

COMMITTEE MEMBERSHIP

TITLE:	Developing a Rapid Urban Forest Assessment System for Sustainable City Greenification
AUTHOR:	Daniel Gonzalez
DATE SUBMITTED:	June 2026
COMMITTEE CHAIR:	Dr. Alex Dekhtyar, Ph.D. Professor of Computer Science
COMMITTEE MEMBER:	Professor Matt Ritter, Ph.D. Professor of Biological Sciences
COMMITTEE MEMBER:	Jenn Yost, Ph.D. Professor of Biological Sciences
COMMITTEE MEMBER:	Professor Jonathan Ventura, Ph.D. Professor of Computer Science

ABSTRACT

Developing a Rapid Urban Forest Assessment System for Sustainable City Greenification

Daniel Gonzalez

This thesis presents the Rapid Urban Forest Assessment (RUFA) system, a web-based platform that integrates urban tree inventories and aerial tree detection to assess forest health across California's census-designated places. RUFA combines inventoried tree records with coordinates detected from high-resolution multispectral imagery using convolutional neural networks, then computes a composite RUFA Score from four metrics: canopy cover percentage, trees per capita, tree diversity (TD-50), and tree evenness. The thesis addresses two engineering challenges in building the dashboard: querying and aggregating over seven million tree records in real time, and rendering spatial summaries at multiple zoom levels without recomputing cluster assignments on every map interaction. A MySQL schema with indexing and cascading materialized views enables sub-second city-level queries, and a SuperCluster index paired with Voronoi tessellation supports interactive map visualization. Deployed on AWS, RUFA provides urban foresters, planners, and researchers with comparable scores and multi-scale map outputs to support evidence-based urban forestry management.

Keywords: urban forestry, tree density, biodiversity, TD-50, scalable, convolutional neural networks

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to the following individuals for their invaluable support and contributions to the development of the RUFA System:

- **Professor Matt Ritter** and **Jenn Yost**, for entrusting me with the UFEI infrastructure and RUFA project. Their confidence and guidance have been instrumental in the success of this research.
- **Dr. Alex Dekhtyar**, for his continuous guidance and insightful feedback throughout the project. His expertise has significantly shaped the direction and quality of this research.
- **Professor Rosa Chang**, **Cameron Squire**, **Dylan Lewis**, **Eric Tong**, **Kristian Markov**, and **Katherin Clark** from San Jose State University, for their dedicated efforts in developing the front-end of RUFA. Their efforts greatly enhanced the functionality and user experience of the assessment system.
- **Cami Pawlak**, for her assistance in providing RUFA data and related formulas, which was essential for the implementation and testing phases of the project.

Additionally, I would like to acknowledge all other individuals and organizations that have supported me throughout this research and development journey.

TABLE OF CONTENTS

Page

LIST OF TABLES ix

LIST OF FIGURES x

CHAPTER

1. INTRODUCTION 1

2. BACKGROUND 2

2.1 Diversity Metrics 2

2.2 Canopy and Density 3

2.3 RUFA Score 4

3. RUFA DATABASE DESIGN 6

3.1 SQL Workflow 6

3.1.1 Tables and Normalization 6

3.1.2 Entity Relationship Diagram 7

3.1.3 Alternative Database Solutions 8

3.2 Materialized Views 10

3.3 Indexing Strategies 10

3.3.1 Automatic Index Maintenance 11

3.4 Materialized View Implementation for RUFA Score Management . . . 11

3.4.1 Overview 11

3.4.2 Geographic Cascade Architecture 12

3.4.2.1 Unified Statistics Table 12

3.4.2.2 Relationship Mapping and Cascade Logic 12

3.4.3 Data Consistency Safeguards 15

3.4.3.1 Version Control 15
3.4.3.2 Atomic Updates 15
3.4.4 Performance Optimization 16
3.4.4.1 Selective Refresh 16
3.4.4.2 Parallel Processing 16
3.4.4.3 Caching Strategy 17
3.4.5 Monitoring and Alerting 17
3.4.6 Indexing Results 17
3.4.7 Performance Comparison Results 18
3.4.7.1 S3 Single CSV File Approach 19
3.4.7.2 S3 Per-City CSV Files 19
3.4.7.3 Database Table Without Indexing 19
3.4.7.4 Database Table With City Indexing 20
3.4.8 Memory and Storage Impact 20
3.4.9 Scaling Considerations 20
4. SYSTEM DESIGN 22
4.1 Infrastructure Used 24
4.1.1 System Diagram 24
4.1.2 Database 24
4.1.3 S3 - ESRI Connectivity 24
4.1.4 Scalability 24
4.2 Security and Permissions 24
4.2.1 User Store 24
4.2.2 Permissions 24
4.2.3 NGINX 24

4.3 Deployment 24
4.3.1 CI/CD 24
5. CLUSTERING 25
5.1 K-Means 25
5.2 K-Means with Caching 26
5.3 Additional Approaches 27
5.3.1 Hierarchical Clustering 27
5.3.2 Grid-Based Clustering 27
5.4 SuperClustering with Voronoi Diagrams 28
5.4.1 System Overview 28
5.4.2 SuperCluster Configuration 28
5.4.3 Voronoi Diagram Generation 29
5.4.4 Spatial Analysis Metrics 30
5.4.5 Point Assignment Algorithm 30
5.4.6 TD-50 Diversity Calculation 31
5.4.7 Performance Characteristics 31
6. CONCLUSIONS 33
6.1 Future Work 35
6.1.1 Allometry Feature 35
6.2 Architecture Changes 35
BIBLIOGRAPHY 37

LIST OF TABLES

Table Page

3.1 Performance comparison of different storage and indexing strategies 18

LIST OF FIGURES

Figure		Page
3.1	Entity Relationship Diagram (ERD) for the RUFA database schema	7

Chapter 1

INTRODUCTION

Urban and community forestry (U&CF) programs play a crucial role in nurturing and enhancing urban green spaces, contributing significantly to the environmental, social, and economic well-being of urban communities. To support these initiatives, advanced tools are being developed to evaluate and report on U&CF activities effectively. One such endeavor is the proposed Urban Forestry Assessment Dashboard by Cal Poly, a web-based platform designed to integrate, analyze, and disseminate comprehensive data on U&CF. This dashboard aims to serve as a centralized hub for urban forestry data collection and analysis, providing actionable insights into the health and management of urban green spaces while seamlessly integrating resources such as those available on https://www.selectree.calpoly.edu. The Urban Forestry Assessment Dashboard will harness these comprehensive datasets, focusing on metrics such as canopy cover, trees per capita, species diversity (via TD-50), and total inventoried trees. By computing dynamic scoring for each city, the system will provide an evolving perspective on urban forestry health and performance. Regular updates and real-time integration with urban tree inventories ensure that this platform remains a valuable and adaptive resource for urban forestry stakeholders.

This initiative not only aims to consolidate urban forestry data into an accessible and actionable format but also seeks to pave the way for innovative and resilient urban forestry management strategies, fostering sustainable urban environments for future generations.

Chapter 2

BACKGROUND

The Rapid Urban Forest Assessment (RUFA) system integrates two primary data sources to provide comprehensive insights into urban forest ecosystems. The first data source offers highly detailed tree characteristics, derived from an extensive compilation of California urban forest inventories. These inventories were aggregated from diverse sources, including private arborist firms (West Coast Arborists, Davey Tree Company, APlus Tree Care) and public entities such as CAL FIRE. This robust dataset ensures a high level of granularity and accuracy in tree attribute information [4]. Our second data source estimates tree coordinates using a deep learning approach. This method employs high-resolution multispectral aerial imagery to detect individual trees in urban settings. A convolutional neural network is utilized to generate a confidence map by regressing the detected tree information, enabling precise spatial quantification of urban tree coverage [10].

2.1 Diversity Metrics

Diversity metrics play a crucial role in understanding the composition and resilience of urban forests. Among these, the TD-50 index, introduced by Love et al. [4], has emerged as a pivotal tool for assessing urban tree species diversity. This index identifies the species that constitute the top 50% of urban forest abundance, providing a snapshot of both ecological dominance and species richness.

The value of such metrics lies in their ability to guide urban forestry programs toward enhanced biodiversity. High diversity levels improve resilience to pests, diseases, and environmental changes, while low diversity often signifies vulnerability. For instance, McPherson et al. [5] highlight the importance of diversifying tree species as a strategy to bolster urban forest resilience against climate change impacts. Furthermore, Livesley et al. [3] emphasize the ecological functions provided by diverse urban forests, such as improved air quality, habitat provision, and the regulation of urban microclimates.

Research underscores that species diversity influences urban forests' ability to adapt to environmental stressors, including extreme weather events and urban heat islands. Therefore, incorporating diversity metrics into urban forestry management is critical for maintaining ecological stability and sustainability.

2.2 Canopy and Density

Urban tree canopy coverage and density are fundamental indicators of urban forest health and performance. Studies like those by Livesley et al. [3] and Sanusi et al. [8] underscore the significance of these metrics in understanding urban tree contributions to ecological and social well-being.

Canopy coverage is particularly effective in regulating urban temperatures and reducing solar radiation. This is especially relevant for small-statured trees, which Sanusi et al. [8] found to be instrumental in urban cooling, despite their limited size. Additionally, canopy density directly correlates with urban areas' capacity to manage stormwater runoff, as shown in Xiao and McPherson's [11] research.

Density metrics also provide insights into forest sustainability. Higher tree densities often indicate robust forest growth and effective carbon sequestration capabilities. However, overcrowding can lead to competition for resources, reducing overall health and resilience. Balancing canopy coverage and density is therefore essential for maximizing the benefits of urban forests, from cooling effects and stormwater management to improving urban aesthetics and human well-being.

2.3 RUFA Score

The Rapid Urban Forest Assessment (RUFA) system also provides a composite "RUFA Score" for each census-designated place in California, facilitating an at-a-glance evaluation of urban forest health and sustainability. This score integrates four equally weighted metrics:

- **Canopy cover percentage (CCP)** – Reflects the proportion of land area covered by tree canopy.
- **Trees per capita (TPC)** – Indicates tree distribution relative to population size.
- **Tree diversity (TD)** – Incorporates the TD-50 index [4] to capture species richness and dominance.
- **Tree evenness (TE)** – Describes how uniformly individuals are distributed among available species.

Each metric is *binned* into a score of 1 to 5 based on its distribution across California. Summing these binned values yields a maximum of 20 possible points. The final RUFA Score is then calculated as:

$$\text{RUFA Score} = \left(\frac{\text{CCP} + \text{TD} + \text{TE} + \text{TPC}}{20} \right) \times 100, \tag{2.1}$$

where higher values (closer to 100) indicate stronger overall performance. By normalizing each component and summing their contributions, the RUFA Score offers a standardized, comparable index of urban forest health. This approach ensures that areas with varying population densities, canopy coverage, and species compositions can be evaluated on a consistent scale, guiding strategic planning and policy decisions to enhance urban forest resilience.

Chapter 3

RUFA DATABASE DESIGN

In this chapter, we delve into the rationale behind the relational database for the RUFA project, a system designed to track tree records across various geographic and ecological dimensions. A relational database provides an ideal solution for managing structured data, such as tree attributes, geographic metrics, and hierarchical relationships between places, zip codes, and tracts. We will explore the MySQL workflow, emphasizing data integrity, normalization principles, materialized views for efficient querying, and indexing strategies to optimize database performance. These features ensure scalable, accurate, and high-performance data management for the RUFA project.

3.1 SQL Workflow

A well-defined SQL workflow is crucial for maintaining data integrity and ensuring efficient data retrieval [2]. The following subsections will detail the tables used in the RUFA database, the normalization process adopted, and the overall database schema design.

3.1.1 Tables and Normalization

The RUFA database consists of several tables designed to minimize redundancy and dependency. By normalizing the database to the third normal form (3NF), we ensure that each table contains data related to a specific topic, thereby enhancing data consistency and simplifying maintenance.

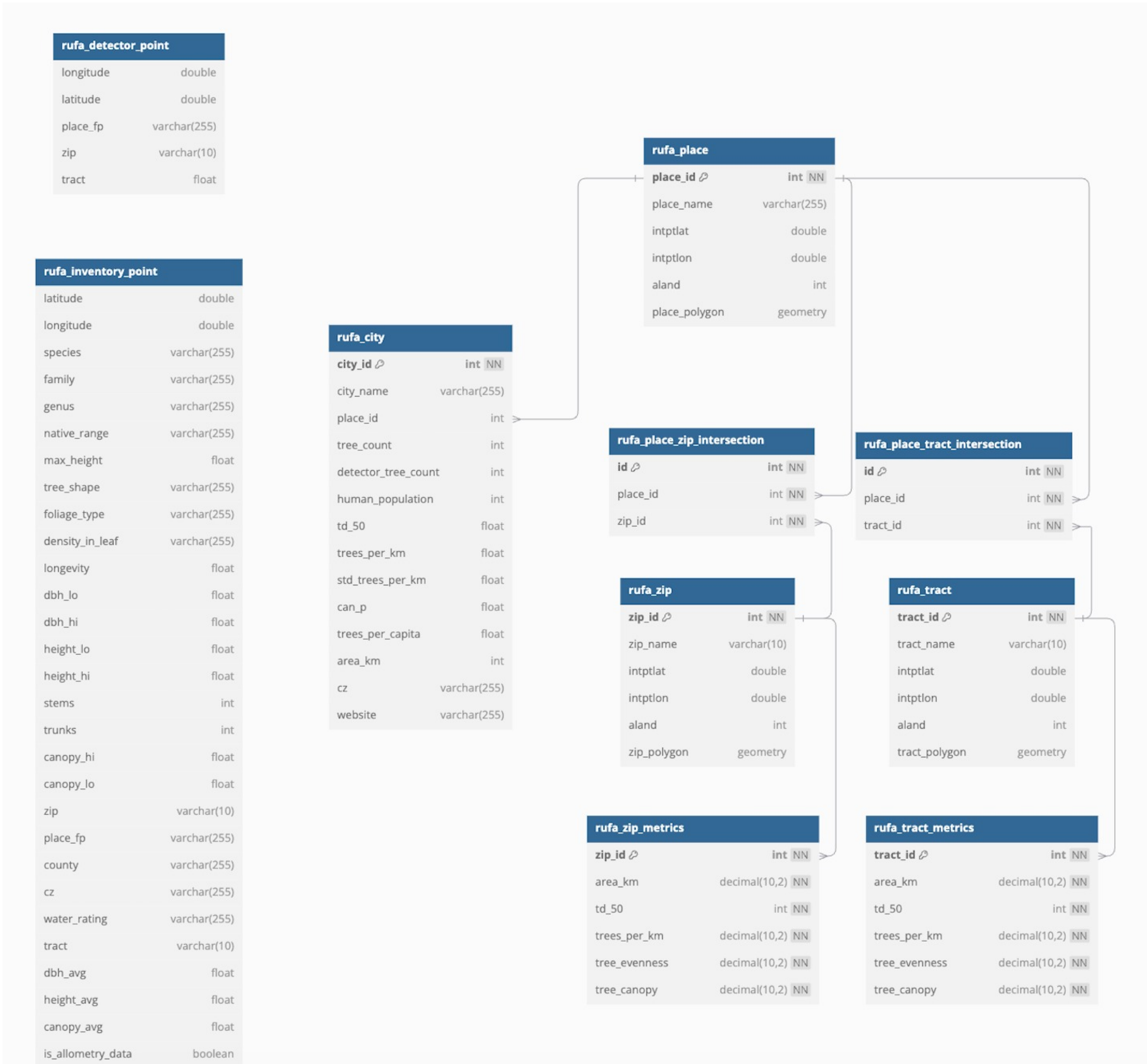

Figure 3.1: *Entity Relationship Diagram (ERD) for the RUFA database schema*

3.1.2 Entity Relationship Diagram

Figure 3.1 illustrates the ERD for the RUFA database schema. This diagram highlights the relationships between different entities, such as Points, Polygons, and Places, ensuring that the database structure supports all necessary operations and queries efficiently. Points are derived from our core data sources, while polygons represent geographic boundaries including census places, ZIP codes, and census tracts.

3.1.3 Alternative Database Solutions

While MySQL serves as the database choice for the RUFA, there are several alternative systems that offer enhanced capabilities for spatial data management and analysis. Given that the RUFA project relies on precise geographical information—such as census tracts, city boundaries, and tree coordinates—optimizing spatial queries and performance can be a critical consideration.

PostgreSQL/PostGIS. One of the most commonly cited alternatives for spatially intensive applications is PostgreSQL coupled with its PostGIS extension [7]. PostGIS extends the relational engine to include advanced geospatial data types, indexing methods (e.g., GiST, SP-GiST), and an extensive library of geospatial functions (for instance, ST_Intersect, S_Buffer, and ST_Distance). These features support complex spatial queries, such as determining which trees fall within a specific bounding polygon or calculating nearest-neighbor distances between tree points. Additionally, PostGIS's robust handling of coordinate reference systems (CRS) and transformations can greatly simplify cross-regional comparisons within the RUFA framework.

MongoDB with Geospatial Indexes. MongoDB, a NoSQL document database, also offers geospatial indexing that can handle queries for distance-based searches on points, lines, and polygons [1]. This schema-free approach can be advantageous for storing heterogeneous tree data—especially when the attributes of different species vary significantly—since new fields can be added without redefining a rigid schema.

GIS-Specific Databases. Some organizations adopt specialized spatial database solutions, such as *Esri's ArcGIS Enterprise* or *Oracle Spatial and Graph*, for high-volume or enterprise-level spatial data management. These systems offer robust toolsets for spatial analyses, but they can come with higher licensing costs and steeper learning curves. For smaller projects or open-source-driven initiatives, this may pose resource constraints and limit broad community involvement.

Why MySQL Remains in Use. Despite these more spatially advanced alternatives, MySQL remains the primary choice for the RUFA project due to its historical ties with the selectree.calpoly.edu infrastructure. Many existing scripts, tools, and user-facing applications are already deeply integrated with MySQL, making a complete or partial transition to another platform expensive in terms of development effort and time. Moreover, while MySQL's native spatial features are comparatively limited, they can still manage a variety of spatial queries by leveraging *SPATIAL indexes* on MyISAM or InnoDB tables.

For MyISAM and InnoDB tables, search operations in columns containing spatial data can be optimized using SPATIAL indexes. The most typical operations are:

- *Point queries* that search for all objects containing a given point
- *Region queries* that search for all objects overlapping a specified region

MySQL uses *R-Trees* with quadratic splitting for SPATIAL indexes on geometry columns. A SPATIAL index is built using the *minimum bounding rectangle* (MBR) of a geometry. For most geometries, the MBR is the smallest rectangle that completely encloses the shape. For a horizontal or vertical linestring, the MBR degenerates to that linestring; and for a point, it degenerates to a single point. It is also possible to create normal (non-SPATIAL) indexes on spatial columns, although these indexes have more restrictions—for instance, in a non-SPATIAL index, you must generally restrict the data to a specific storage engine or data type to ensure efficient lookups.

Ultimately, although PostgreSQL/PostGIS/MongoDB with geospatial indexes, or specialized GIS databases may outperform MySQL in certain spatial workflows, the RUFA project's current operational environment and alignment with Selectree's architecture underscore MySQL's continued use. Future iterations of RUFA could incorporate or migrate to these alternatives if spatial demands grow significantly—potentially leveraging hybrid architectures that pair MySQL's relational strengths with specialized spatial extensions in a multi-database ecosystem.

3.2 Materialized Views

Materialized views are used in the RUFA database to store the results of complex queries, thereby improving query performance. To ensure that these views remain up-to-date with the underlying data, we implement automatic refresh mechanisms. This can be achieved using database triggers or scheduled jobs that periodically update the materialized views based on changes in the base tables.

3.3 Indexing Strategies

Effective indexing is essential for optimizing query performance in the RUFA database. By carefully selecting which columns to index, we can significantly reduce query execution time. Automatic updates to indexes are managed by the database system, which maintains the indexes as data is inserted, updated, or deleted. Additionally, regular index maintenance tasks, such as rebuilding fragmented indexes, are scheduled to ensure optimal performance.

3.3.1 Automatic Index Maintenance

Modern relational database management systems (RDBMS) handle index maintenance automatically. However, for large databases like RUFA, it's advisable to monitor index performance and perform manual maintenance when necessary. Tools and scripts can be employed to automate these maintenance tasks, ensuring that indexes remain efficient without manual intervention.

3.4 Materialized View Implementation for RUFA Score Management

3.4.1 Overview

The RUFA system relies on complex calculations involving multiple metrics—canopy cover percentage (CCP), trees per capita (TPC), tree diversity (TD-50), and tree evenness (TE)—to generate composite RUFA scores for census-designated places across California. Given the computational intensity of these calculations and the need for responsive user queries, implementing materialized views becomes essential for maintaining system performance while ensuring data accuracy.

The system faces a unique challenge in that geographic areas are hierarchically related—when tree data changes in a city like Los Angeles, the system must update not only the city-level statistics but also all associated ZIP codes, census tracts, climate zones, and county-level aggregations. This cascading update requirement necessitates a sophisticated materialized view strategy that can efficiently propagate changes through the geographic hierarchy.

3.4.2 Geographic Cascade Architecture

Since MariaDB does not natively support true materialized views like PostgreSQL, the RUFA system implements a specialized architecture using regular tables that function as materialized views, combined with automated refresh mechanisms to maintain data consistency across geographic hierarchies.

The core innovation lies in the cascading update system that recognizes the interconnected nature of geographic boundaries. When tree inventory data changes in any location, the system automatically identifies and updates all related geographic areas through a relationship mapping table and stored procedure framework. This ensures that statistics remain synchronized across ZIP codes, cities, census tracts, climate zones, and counties without requiring manual intervention.

3.4.2.1 Unified Statistics Table

The system maintains a central `tree_stats_by_location` table that stores pre-calculated metrics for all geographic location types within a single, normalized structure. This table contains comprehensive tree statistics including species diversity counts, average tree characteristics, native species distributions, and water requirement breakdowns. Each record is identified by a location type (zip, city, tract) and corresponding location code, enabling efficient querying across different geographic scales.

3.4.2.2 Relationship Mapping and Cascade Logic

A separate `location_relationships` table maintains the hierarchical connections between different geographic boundaries, enabling the system to determine which areas need updates when tree data changes. The cascade logic is implemented through stored procedures that automatically refresh statistics for all related geographic areas when triggered by data modifications.

For example, when tree inventory data changes in Los Angeles, the system automatically updates statistics for the city itself, all ZIP codes within Los Angeles boundaries, associated census tracts, the relevant climate zone, and Los Angeles County. This ensures consistency across all geographic scales without requiring users or administrators to manually trigger updates for each affected area.

Trigger-Based Updates Database triggers are implemented on core tables to automatically invalidate and schedule refreshes of dependent materialized views:

Code Listing 3.1: *Database trigger for automatic refresh scheduling*

```
DELIMITER $
CREATE TRIGGER update_rufa_scores_trigger
AFTER INSERT ON tree_inventory
FOR EACH ROW
BEGIN
    UPDATE mv_refresh_schedule
    SET needs_refresh = TRUE,
        last_modified = NOW()
    WHERE view_name = 'mv_rufa_scores'
    AND place_id = NEW.place_id;
END$
DELIMITER ;
```

Scheduled Batch Processing A scheduled job runs every N hours to process bulk updates and recalculate scores for modified areas:

Code Listing 3.2: *Batch refresh procedure for RUFA scores*

```
-- Batch refresh procedure
DELIMITER $
CREATE PROCEDURE RefreshRUFAScores()
BEGIN
    DECLARE done INT DEFAULT FALSE;
```

```
    DECLARE place_to_update VARCHAR(50);

    -- Cursor for places needing updates
    DECLARE place_cursor CURSOR FOR
        SELECT DISTINCT place_id
        FROM mv_refresh_schedule
        WHERE needs_refresh = TRUE;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done =
       TRUE;

    OPEN place_cursor;

    refresh_loop: LOOP
        FETCH place_cursor INTO place_to_update;
        IF done THEN
            LEAVE refresh_loop;
        END IF;

        -- Recalculate RUFA score components
        CALL CalculateRUFAScore(place_to_update);

        -- Mark as refreshed
        UPDATE mv_refresh_schedule
        SET needs_refresh = FALSE,
            last_refreshed = NOW()
        WHERE place_id = place_to_update;

    END LOOP;

    CLOSE place_cursor;
END$
DELIMITER ;
```

Incremental Update Strategy Rather than full table refreshes, the system implements incremental updates that only recalculate scores for affected geographic areas:

- **Spatial Partitioning**: RUFA scores are partitioned by climate zones and counties to isolate updates

- **Change Detection**: Modified tree records trigger updates only for their specific census-designated places
- **Dependency Tracking**: A dependency graph ensures that changes to base data propagate correctly through all dependent materialized views

3.4.3 Data Consistency Safeguards

To maintain data integrity during the refresh process:

3.4.3.1 Version Control

Each materialized view maintains version numbers to detect stale data:

Code Listing 3.3: *Version control table for materialized views*

```
CREATE TABLE mv_version_control (
    view_name VARCHAR(50) PRIMARY KEY,
    current_version INT NOT NULL DEFAULT 1,
    last_refresh TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    refresh_duration_seconds INT,
    status ENUM('CURRENT', 'REFRESHING', 'STALE')
       DEFAULT 'CURRENT'
);
```

3.4.3.2 Atomic Updates

Large refresh operations use temporary tables and atomic swaps to prevent serving inconsistent data:

Code Listing 3.4: *Atomic table swap for consistent updates*

```
-- Create temporary table with updated scores
CREATE TABLE mv_rufa_scores_temp LIKE mv_rufa_scores;
```

```sql
-- Populate with fresh calculations
INSERT INTO mv_rufa_scores_temp
SELECT place_id,
       ((ccp_score + td_score + te_score + tpc_score) /
          20) * 100 as rufa_score,
       NOW() as calculated_at
FROM mv_tree_metrics
JOIN mv_population_data USING (place_id);

-- Atomic swap
RENAME TABLE mv_rufa_scores TO mv_rufa_scores_old,
             mv_rufa_scores_temp TO mv_rufa_scores;

DROP TABLE mv_rufa_scores_old;
```

3.4.4 Performance Optimization

The materialized view system includes several performance optimizations:

3.4.4.1 Selective Refresh

Only geographic areas with actual data changes are recalculated, dramatically reducing refresh times from hours to minutes for typical updates.

3.4.4.2 Parallel Processing

Multiple refresh workers can simultaneously update different geographic partitions, leveraging MariaDB's parallel processing capabilities.

3.4.4.3 Caching Strategy

Frequently accessed RUFA scores are cached in Redis with appropriate TTL values, reducing database load for dashboard queries.

3.4.5 Monitoring and Alerting

The system includes comprehensive monitoring to ensure materialized views remain current:

- **Staleness Detection**: Automated alerts when materialized views fall behind source data by more than 12 hours
- **Performance Monitoring**: Tracking of refresh times and resource utilization
- **Data Quality Checks**: Validation that recalculated scores fall within expected ranges

This materialized view architecture ensures that RUFA scores remain accurate and current while providing the query performance necessary for responsive user interactions with the Urban Forestry Assessment Dashboard.

3.4.6 Indexing Results

To evaluate the performance impact of different data storage and indexing strategies, we conducted comprehensive benchmarking tests across various query scenarios. Our analysis compared four different approaches for data retrieval and rendering within the RUFA system, measuring query execution times and system responsiveness under typical usage patterns. All performance tests were conducted on a standardized test environment consisting of an AWS EC2 instance (t3.large) with 2 vCPUs, 8GB RAM, and 10.5.23 MariaDB running on Amazon RDS. The test dataset contained approximately 7.2 million tree records distributed across 702 cities in California, matching the production data scale described in the California Urban Forest Inventory.

Each test scenario was executed 100 times to ensure statistical significance, with query execution times measured using MySQL's built-in profiling tools. The performance metrics were calculated as:

$$\text{Throughput} = \frac{1}{\text{Average Query Time (seconds)}} \tag{3.1}$$

$$\text{Performance Improvement} = \frac{T_{\text{baseline}} - T_{\text{optimized}}}{T_{\text{baseline}}} \times 100\% \tag{3.2}$$

where T_{baseline} represents the baseline query time and $T_{\text{optimized}}$ represents the optimized query time.

3.4.7 Performance Comparison Results

Table 3.1 presents the average query execution times across different storage and indexing strategies for city-specific queries retrieving tree inventory data.

Table 3.1: *Performance comparison of different storage and indexing strategies*

Storage Method	Avg. Time (ms)	Std. Dev.	Queries/s
S3 Single CSV File	8,450	1,230	0.12
S3 Per-City CSV Files	3,120	480	0.32
Database Table (No Index)	2,890	290	0.35
Database Table (Index)	145	25	6.90

3.4.7.1 S3 Single CSV File Approach

The initial approach of storing all tree data in a single CSV file on Amazon S3 demonstrated the poorest performance, with average query times exceeding 8.4 seconds. This method required downloading and parsing the entire dataset for each query, regardless of the specific city or region being requested. The high standard deviation ($\sigma = 1,230$ ms) indicated inconsistent performance, likely due to network latency variations and the overhead of processing large file transfers.

3.4.7.2 S3 Per-City CSV Files

Partitioning the data into individual CSV files per city showed significant improvement, reducing average query times to 3.12 seconds—a 63% performance improvement over the single file approach. This approach eliminated the need to process irrelevant data for city-specific queries. However, the system still faced overhead from file system operations and CSV parsing, limiting its scalability for real-time applications.

3.4.7.3 Database Table Without Indexing

Direct queries against the MySQL database table without proper indexing yielded slightly better performance than the partitioned CSV approach, with average query times of 2.89 seconds. While this eliminated file parsing overhead, the lack of indexing on frequently queried columns (particularly city names and geographic identifiers) required full table scans with $O(n)$ complexity for most operations.

3.4.7.4 Database Table With City Indexing

The implementation of a B-tree index on the city column resulted in dramatic performance improvements, reducing average query times to just 145 milliseconds—a 95% improvement over the non-indexed approach. The low standard deviation ($\sigma = 25$ ms) demonstrates consistent performance across different query patterns. This approach achieved a throughput of 6.90 queries per second, making it suitable for interactive web applications and real-time dashboard updates.

3.4.8 Memory and Storage Impact

The indexing strategy increased database storage requirements by approximately 15%, from 2.3GB to 2.6GB for the complete dataset. However, this storage overhead was offset by significant reductions in memory usage during query execution, as indexed queries required substantially less working memory for result set processing.

The storage overhead can be expressed as:

$$\text{Storage Overhead} = \frac{S_{\text{indexed}} - S_{\text{original}}}{S_{\text{original}}} \times 100\% = \frac{2.6 - 2.3}{2.3} \times 100\% = 13.0\% \quad (3.3)$$

3.4.9 Scaling Considerations

Performance testing with varying dataset sizes demonstrated that the indexed approach maintains logarithmic time complexity $O(\log n)$, ensuring scalability as the urban forest inventory continues to grow. Projections suggest that even with a 10× increase in data volume, indexed query times would remain under 300ms for typical city-based queries, maintaining the system's responsiveness for interactive applications.

Chapter 4

SYSTEM DESIGN

The rapid urban forest assessment system requires a robust and scalable architecture capable of handling large-scale tree inventory data while providing real-time query capabilities for municipal planners and environmental researchers. Building upon the performance analysis presented in Chapter 3, which demonstrated that indexed database queries achieve 95% performance improvements over non-indexed approaches, this chapter outlines the comprehensive system design that enables efficient processing and visualization of urban forest data across California's 702+ cities.

The system architecture addresses three critical design requirements: scalability to accommodate the 7.2 million tree records in the California Urban Forest Inventory, performance optimization to support interactive web applications with sub-second response times, and extensibility to incorporate additional environmental datasets and analysis capabilities. The design leverages cloud-native technologies and modern web frameworks to create a maintainable and cost-effective solution.

This chapter is organized into three main sections that detail the key architectural components. Section 4.1 presents the infrastructure utilized, including a comprehensive system diagram, the optimized database design that achieves 6.90 queries per second throughput, S3 integration with ESRI connectivity for geospatial data processing, and scalability considerations for future growth. Section 4.2 outlines the security and permissions framework, covering user store management, role-based permissions architecture, and NGINX configuration for secure request handling. Finally, Section 4.3 describes the deployment strategy, focusing on the continuous integration and continuous deployment (CI/CD) pipeline that ensures reliable system updates and maintenance.

The resulting system design provides a foundation for sustainable city greenification efforts by enabling rapid assessment of urban forest conditions, identification of tree planting opportunities, and data-driven decision making for municipal environmental programs.

4.1 Infrastructure Used

4.1.1 System Diagram

4.1.2 Database

4.1.3 S3 - ESRI Connectivity

4.1.4 Scalability

4.2 Security and Permissions

4.2.1 User Store

4.2.2 Permissions

4.2.3 NGINX

4.3 Deployment

4.3.1 CI/CD

Chapter 5

CLUSTERING

The clustering component of the RUFA system provides efficient spatial grouping of urban tree data to enable rapid analysis and visualization at various geographic scales. This chapter presents the evolution of clustering algorithms implemented, from basic K-means to advanced SuperClustering with Voronoi diagrams, demonstrating progressive performance improvements for handling California's 6.6 million tree inventory records.

5.1 K-Means

The initial clustering implementation utilized the traditional K-means algorithm for spatial grouping of tree coordinates. K-means partitions the dataset into k clusters by minimizing the within-cluster sum of squares (WCSS):

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2 \tag{5.1}$$

where C_i represents cluster i, μ_i is the centroid of cluster i, and x represents individual tree coordinates.

Implementation Details: The basic K-means implementation processes tree coordinates using the Lloyd's algorithm with the following specifications:

- **Distance Metric:** Haversine distance for geographic coordinates

- **Initialization:** K-means++ for optimal initial centroid selection
- **Convergence Criteria:** Maximum 100 iterations or centroid movement ¡ 0.001°
- **Cluster Count:** Dynamic k selection based on zoom level (k = 50-500)

Performance Characteristics: Initial benchmarking revealed significant computational overhead for large datasets:

- Average execution time: 2.3 seconds for 100,000 trees
- Memory usage: 45MB for coordinate processing
- Scalability limitation: O(nkt) complexity where n=points, k=clusters, t=iterations

The basic K-means approach provided adequate clustering quality but suffered from performance issues when processing the full California dataset, necessitating optimization strategies.

5.2 K-Means with Caching

To address performance limitations, the second iteration implemented comprehensive caching mechanisms for both intermediate computations and final cluster results.

Caching Strategy:

- **Centroid Cache:** Cached cluster centroids across repeated queries for the same viewport

Caching reduced redundant computation when users returned to previously viewed regions, but did not address the cost of recomputing clusters at each zoom level.

5.3 Additional Approaches

Recognizing the limitations of standard K-means for geographic data, several alternative clustering approaches were evaluated and implemented.

5.3.1 Hierarchical Clustering

Agglomerative hierarchical clustering was implemented to create multi-level tree groupings supporting zoom-dependent visualizations:

- **Linkage Criteria:** Ward linkage minimizing within-cluster variance
- **Distance Matrix:** Sparse matrix optimization for geographic coordinates
- **Dendrogram Pruning:** Dynamic tree cutting based on zoom level requirements

Performance analysis showed hierarchical clustering provided better multi-scale representations but with increased computational complexity $O(n^3)$, making it suitable only for smaller regional datasets.

5.3.2 Grid-Based Clustering

A custom grid-based approach was another approach for rapid initial clustering:

- **Grid Resolution:** Adaptive cell size based on tree density (0.001° - 0.01°)
- **Spatial Index:** R-tree spatial indexing for efficient range queries
- **Aggregation:** Statistical summaries computed per grid cell

Grid-based clustering achieved O(n) complexity but lacked the flexibility needed for variable-density urban environments.

5.4 SuperClustering with Voronoi Diagrams

This clustering implementation combines hierarchical clustering with Voronoi diagram generation to analyze urban tree data efficiently. The approach creates spatial boundaries at multiple zoom levels, enabling both detailed local analysis and broader regional insights.

5.4.1 System Overview

The system integrates two key technologies:

- **SuperCluster**: Hierarchical point clustering across zoom levels 0–10
- **Voronoi Diagrams**: Spatial tessellation for regional boundary creation

This combination enables multi-scale spatial analysis with efficient point aggregation and real-time visualization capabilities.

5.4.2 SuperCluster Configuration

The clustering process uses pysupercluster with the following parameters:

Code Listing 5.1: *Basic SuperCluster Setup*

```
# Initialize clustering with key parameters
index = pysupercluster.SuperCluster(
    detector_points,
    min_zoom=0,
```

```
        max_zoom=10,
        radius=5,
        extent=512
    )

    # Process zoom levels
    for zoom in range(5, 10):
        clusters = index.getClusters(bbox, zoom)
        if len(clusters) >= 4:
            voronoi_data = process_voronoi(clusters, zoom)
```

Key Configuration Parameters:

Zoom Range 0 to 10 levels for hierarchical analysis

Cluster Radius 5-pixel aggregation radius

Tile Extent 512-pixel spatial indexing

Coverage Global bounding box (-180°, 90°) to (180°, -90°)

5.4.3 Voronoi Diagram Generation

For each zoom level with sufficient cluster points, Voronoi diagrams create spatial boundaries:

Code Listing 5.2: *Simplified Voronoi Processing*

```
def process_voronoi(cluster_points, zoom):
    # Generate Voronoi diagram
    vor = Voronoi(cluster_points)

    # Create polygons from Voronoi edges
    polygons = create_polygons_from_voronoi(vor)

    # Process each Voronoi cell
    for polygon in polygons:
        # Calculate area and tree metrics
        area_km = calculate_area(polygon)
```

```
        trees_in_cell = count_trees_in_polygon(polygon)

        # Compute diversity and density
        tree_density = trees_in_cell / area_km
        diversity_score = calculate_td_50(polygon)

        # Add to results with color mapping
        add_feature_to_geojson(polygon, tree_density,
           diversity_score)
```

5.4.4 Spatial Analysis Metrics

Each Voronoi cell generates comprehensive urban forestry metrics:

Tree Density Trees per square kilometer within each cell

Species Diversity TD-50 index measuring ecological diversity

Color Mapping Normalized visualization based on density values

Geometric Validation Ensures valid polygon output for mapping

5.4.5 Point Assignment Algorithm

Trees are efficiently assigned to Voronoi cells using spatial containment:

Code Listing 5.3: *Point-in-Polygon Assignment*

```
def assign_trees_to_cells(points, voronoi_polygons):
    for polygon in voronoi_polygons:
        trees_in_cell = []
        for tree_point in points:
            if polygon.contains(tree_point):
                trees_in_cell.append(tree_point)

        # Calculate metrics for this cell
        process_cell_metrics(polygon, trees_in_cell)
```

5.4.6 TD-50 Diversity Calculation

The TD-50 index measures species diversity within each Voronoi cell:

Code Listing 5.4: *Regional Diversity Assessment*

```
def calculate_td_50(species_counts):
    if not species_counts:
        return 0

    # Sort species by abundance
    sorted_counts = sorted(species_counts, reverse=True
       )
    total_trees = sum(sorted_counts)

    # Find species needed for 50% of trees
    cumulative = 0
    for i, count in enumerate(sorted_counts):
        cumulative += count
        if cumulative >= total_trees * 0.5:
            return i + 1
```

5.4.7 Performance Characteristics

The SuperClustering implementation demonstrates significant computational efficiency:

Algorithmic Complexity:

- SuperCluster initialization: $O(n \log n)$
- Voronoi generation: $O(n \log n)$ using Fortune's algorithm
- Point assignment: $O(kn)$ where k is the number of cells
- Overall complexity: $O(n \log n + kn)$

Multi-Scale Benefits:

- Higher zoom levels provide detailed local analysis
- Lower zoom levels enable regional pattern recognition
- Cached results allow rapid zoom-level switching
- Hierarchical structure supports interactive exploration

Output Features:

- Valid GeoJSON format for web mapping integration
- Color-coded visualization based on tree density
- Species diversity metrics for ecological assessment
- Scalable performance across California's metropolitan regions

This approach successfully combines the hierarchical efficiency of SuperCluster with the spatial analysis power of Voronoi diagrams, enabling comprehensive urban forestry assessment at multiple scales.

Chapter 6

CONCLUSIONS

The development of the Rapid Urban Forest Assessment (RUFA) system represents a significant advancement in urban forestry management and evaluation. This thesis has presented a comprehensive approach to creating an automated tree assessment system that integrates aerial imagery analysis with extensive urban tree inventories to provide scalable and accurate urban forest assessments for California's census-designated places.

The RUFA system successfully addresses several critical challenges in urban forestry management. By leveraging convolutional neural networks to extract tree coordinates from high-resolution multispectral aerial imagery and combining this with detailed tree inventory data from multiple sources, the system provides a robust foundation for urban forest evaluation. The integration of diverse data sources, including inventories from private arborist firms such as West Coast Arborists, Davey Tree Company, and APlus Tree Care, alongside public entities like CAL FIRE, ensures comprehensive coverage and high data quality.

The system's composite scoring methodology, which incorporates four equally weighted metrics—canopy cover percentage, trees per capita, tree diversity (TD-50), and tree evenness—provides urban planners and forestry professionals with an intuitive and actionable assessment tool. The RUFA Score formula, which normalizes these components to create a standardized index ranging from 0 to 100, enables meaningful comparisons across different geographic areas and varying urban contexts.

The database design choices, particularly the decision to maintain MySQL compatibility with the existing selectree.calpoly.edu infrastructure, demonstrate the importance of balancing technological advancement with practical implementation considerations. While more advanced spatial database solutions like PostgreSQL/PostGIS offer enhanced geospatial capabilities, the RUFA system's architecture prioritizes integration with established workflows and existing tools used by urban forestry professionals.

The implementation of materialized views and strategic indexing approaches ensures that the system can handle large-scale data processing efficiently while maintaining real-time responsiveness for user queries. This performance optimization is crucial for supporting the system's intended use by urban planners, foresters, and environmental agencies who require timely access to assessment results.

The RUFA system's outputs provide valuable insights for Urban and Community Forestry (U&CF) programs, enabling evidence-based decision-making for urban forest management. The percentage-based scores for key metrics allow stakeholders to quickly identify areas requiring attention and track improvements over time. This capability is particularly important as cities increasingly recognize the critical role of urban forests in providing ecosystem services, mitigating urban heat islands, and supporting community well-being.

The success of this project demonstrates the potential for technology to enhance traditional forestry practices while maintaining accessibility and practical utility for end users. Future enhancements will continue to build upon this foundation, expanding the system's capabilities while preserving its core mission of supporting effective urban forest management through accurate, timely, and actionable assessments.

6.1 Future Work

While the current RUFA system provides a solid foundation for urban forest assessment, several areas present opportunities for enhancement and expansion.

6.1.1 Allometry Feature

A significant enhancement planned for future versions of RUFA involves the integration of allometric equations to improve tree estimations. UFEI's AllomeTree tool [9] exposes species-specific equations for California's urban forest, predicting DBH, height, and canopy width from either DBH or tree age. Integrating these models would let RUFA estimate structural attributes for aerial-only detections and extend reporting to ecosystem services such as carbon storage and stormwater interception.

6.2 Architecture Changes

Several architectural improvements could enhance the system's scalability and functionality:

Real-time Data Integration: Incremental materialized-view refresh triggered by inventory updates or new aerial detections would keep RUFA Scores current without recomputing every dependent view.

User Interface Expansion: Development of mobile applications and enhanced web interfaces to support field data collection and real-time assessment updates. This would enable urban foresters to contribute data directly during field surveys and access RUFA assessments from mobile devices.

Multi-State Expansion: Extension of the RUFA system beyond California to support urban forest assessment in other states and regions. This would require adaptation of the assessment criteria and database schema to accommodate different climate zones, species compositions, and regulatory frameworks.

The RUFA system represents a significant step forward in urban forest assessment technology. Its combination of automated analysis, comprehensive data integration, and user-friendly output provides urban forestry professionals with powerful tools for evidence-based decision-making. As urban areas continue to grow and face increasing environmental challenges, systems like RUFA will play an increasingly important role in supporting sustainable urban development and community well-being.

BIBLIOGRAPHY

[1] R. Cattell. "Scalable SQL and NoSQL data stores". In: *ACM SIGMOD Record* 39.4 (2011), pp. 12–27. DOI: 10.1145/1978915.1978919.

[2] E. F. Codd. "A Relational Model of Data for Large Shared Data Banks". In: *Communications of the ACM* 13.6 (1970), pp. 377–387. DOI: 10.1145/362384.362685.

[3] S. J. Livesley, E. G. McPherson, C. Calfapietra, et al. "The Urban Forest and Ecosystem Services: Impacts on Urban Water, Heat, and Pollution Cycles at the Tree, Street, and City Scale". In: *Journal of Urban Ecology* (2016).

[4] N. L. R. Love and E. G. McPherson. "Diversity and structure in California's urban forest". In: *Urban Forestry & Urban Greening* (2022).

[5] E. G. McPherson, A. M. Berry, S. M. L. Studer, et al. "Performance Testing to Identify Climate-Ready Trees for Central Valley Communities". In: *Arboriculture & Urban Forestry* 44.2 (2018), pp. 83–99. DOI: 10.48044/jauf.2018.008.

[6] E. G. McPherson, N. S. van Doorn, and P. J. Peper. *Urban Tree Database and Allometric Equations*. Gen. Tech. Rep. PSW-GTR-253. Albany, CA: U.S. Department of Agriculture, Forest Service, Pacific Southwest Research Station, 2016, p. 86. DOI: 10.2737/RDS-2016-0005.

[7] P. Ramsey and R. Research. *PostGIS Documentation*. Available at https://postgis.net. 2005.

[8] D. J. Sanusi et al. "Street orientation and side of the street greatly influence the microclimatic benefits street trees". In: *Journal of Environmental Quality* (2016).

[9] Urban Forest Ecosystems Institute. *AllomeTree: Allometric Equations for California's Urban Forest*. https://calpolyufei.github.io/allometree-ufei/. Cal Poly Urban Forest Ecosystems Institute.

[10] J. Ventura, C. Pawlak, M. Honsberger, et al. "Individual Tree Detection in Large-Scale Urban Environments using High-Resolution Multispectral Imagery". In: *arXiv* (2022). Available at https://arxiv.org/abs/2208.10607v4.

[11] Q. Xiao and E. G. McPherson. "Surface water storage capacity of twenty tree species in Davis, California. Journal of Environmental Quality". In: *Journal of Environmental Quality* (2016).